🚀 Collections Framework Masterclass

Chapter 16 — LinkedHashMap — Complete Deep Dive

> **Interview Importance:** ⭐⭐⭐⭐⭐

`LinkedHashMap` is essentially:

> **HashMap + predictable ordering**

The key idea is simple:

```
HashMap
   +
Doubly linked list
   ↓
LinkedHashMap
```

It gives you the fast average lookup characteristics of `HashMap` while maintaining a predictable iteration order.

The two modes you must know for interviews are:

1. **Insertion order**
2. **Access order**

The second one leads directly to the classic **LRU Cache** interview problem.

---

1. What is LinkedHashMap?

Package:

```java id="bq8zpd" java.util.LinkedHashMap
```

```
Hierarchy:

```text id="8m3w2x"
Map
```
```

```
 ↓
HashMap
 ↓
LinkedHashMap
```

Conceptually:

```
HashMap
  ↓
Hash table
  +
Linked ordering structure
```

So LinkedHashMap inherits HashMap's basic hashing behavior and adds ordering information.

---

2. Why Does LinkedHashMap Exist?

Suppose you use:

```java id="v3ey2p"
Map<Integer, String> map = new HashMap<>();
```

```
You cannot rely on iteration order.

If you need:

> "Give me entries in the order they were inserted."

Use:

```java id="r7f1n8"
Map<Integer, String> map =
    new LinkedHashMap<>();
```

Example:

```java id="yyjz6k"
map.put(3, "C");
map.put(1, "A");
map.put(2, "B");
```

```
Iteration is predictably:

```text id="e4wz3s"
3 → C
```

```
1 → A
2 → B
```

---

3. LinkedHashMap Internal Structure

This is the most important internal concept.

A normal HashMap has:

```text id="e6v7xw"
table ↓
bucket → Node → Node
```

```
LinkedHashMap additionally maintains links between entries:

```text id="m2f7q0"
Hash table

       +

Doubly linked list
```

Conceptually:

```text id="d1wzqm"
head ↓
[A] ⇄ [B] ⇄ [C] ⇄ [D]
                  ↑ tail
```

```
Each entry participates in:

1. Hash table structure
2. Linked ordering structure

---

# 4. LinkedHashMap Entry

HashMap's basic node contains:

```text id="l6x9kd"
hash
key
value
next
```

LinkedHashMap adds ordering links conceptually:

```text id="6m9c0h" before after
```

````
So an entry can be thought of as:

```text id="7j5r5c"
      Entry
   ┌──────────────┐
   │ hash         │
   │ key          │
   │ value        │
   │ next         │
   │ before       │
   │ after        │
   └──────────────┘
````

The `next` relationship is associated with the hash bucket.

`before` and `after` maintain the linked ordering.

---

5. Two Structures at the Same Time

This is the key to understanding LinkedHashMap.

Suppose:

```java id="j8v1p6" map.put(10, "A"); map.put(20, "B"); map.put(30, "C");
```

````
Hash structure:

```text id="4bl0t8"
bucket
  ↓
entries distributed according to hash
````

Ordering structure:

```text id="9w2t7d" 10 ⇄ 20 ⇄ 30
```

```
Therefore:

```text id="nh9z4s"
Hash table
→ fast lookup

Linked list
→ predictable iteration
```

---

6. Insertion Order

Default constructor:

```java id="glgk9r" Map<Integer, String> map = new LinkedHashMap<>();

```
uses insertion-order behavior.

Example:

```java id="jszv7b"
map.put(10, "A");
map.put(20, "B");
map.put(30, "C");
```

Iteration:

```text id="v5yy7w" 10 20 30

```
---

7. Updating an Existing Key

Important interview detail.

```java id="r4a9hy"
map.put(10, "A");
map.put(20, "B");
map.put(30, "C");

map.put(20, "Updated");
```
```

Does `20` move to the end?

No, in normal insertion-order mode.

Result:

```text id="9c0v4q"
10 20 30
```

```
Only the value changes.

---

# 8. Why Doesn't Updating Move It?

Because the key already exists.

In insertion-order mode:

> The ordering represents the order in which entries were inserted, not the
order in which values were updated.

So:

```java id="0p7px9"
map.put(existingKey, newValue);
```

doesn't normally change its position.

---

9. Access Order

LinkedHashMap has another mode:

```java id="l6o2xk"
accessOrder = true
```

```
Constructor:

```java id="5hx8n3"
LinkedHashMap<Integer, String> map =
    new LinkedHashMap<>(
        16,
        0.75f,
```

```
        true
    );
```

The last argument means:

```text id="j0h5r5" true → access-order false → insertion-order

```
Default:

```text id="a9w3pc"
false
```

---

10. Insertion Order vs Access Order

Insertion order

```text id="h3w6jc" 10 20 30

get(10)

10 20 30

```
Access order

Initially:

```text id="z8d0nq"
10
20
30
```

Then:

```java id="qg4c5r" map.get(10);

```
Order becomes:

```text id="j0q0yy"
20
```

```
30
10
```

The accessed entry moves to the end.

This is the foundation of LRU caches.

---

11. What Counts as an Access?

In access-order mode, several Map operations can count as an access.

Common examples include:

```text id="c9s1aq"
get()
getOrDefault()
putIfAbsent()
replace()
compute()
computeIfAbsent()
computeIfPresent()
merge()
```

```
depending on whether the operation actually accesses/replaces an existing
mapping according to the API semantics.

The important interview takeaway:

> **Access-order LinkedHashMap can reorder entries as a result of map
operations.**

---

# 12. `get()` Can Modify LinkedHashMap

This surprises many candidates.

With:

```java id="9b6i4z"
accessOrder = true
```

calling:

```java id="c4e4ce"
map.get(key);
```

```
can move the accessed entry to the tail.

Therefore, unlike an ordinary HashMap lookup:
```
```

```
> A `get()` can cause a structural ordering change in access-order
LinkedHashMap.

This matters during iteration.

---

# 13. Example

```java id="i0i6e6"
LinkedHashMap<Integer, String> map =
    new LinkedHashMap<>(
        16,
        0.75f,
        true
    );

map.put(1, "A");
map.put(2, "B");
map.put(3, "C");

map.get(1);
```

Before:

```text id="j9t6lq"
1 → 2 → 3
```

```
After:

```text id="lq4x9e"
2 → 3 → 1
```

---

14. LRU Cache

LRU means:

> **Least Recently Used**

Suppose cache capacity is 3:

```text id="p6f0f4"
[A] [B] [C]
```

```
Access A:

```text id="j0g9yo"
[B] [C] [A]
```

Now insert D:

```text id="d3w2ph" [C] [A] [D]
```

```
B was least recently used, so it was removed.

LinkedHashMap's access-order mode makes this pattern extremely convenient.

---

# 15. Building an LRU Cache

Classic implementation:

```java id="j3x3d6"
class LRUCache<K, V>
        extends LinkedHashMap<K, V> {

    private final int capacity;

    public LRUCache(int capacity) {
        super(
            capacity,
            0.75f,
            true
        );

        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(
            Map.Entry<K, V> eldest) {

        return size() > capacity;
    }
}
```

Usage:

```java id="o3d9jh" LRUCache<Integer, String> cache = new LRUCache<>(3);

cache.put(1, "A"); cache.put(2, "B"); cache.put(3, "C");

cache.get(1);

cache.put(4, "D");

```
The least recently used entry is automatically removed.

---

16. `removeEldestEntry()`

This method is the key to the LRU implementation.

```java id="a1y0g8"
protected boolean removeEldestEntry(
    Map.Entry<K,V> eldest)
```

If it returns:

```text id="q4qax3" true

```
the eldest entry is removed after an insertion.

For an LRU cache:

```java id="u3y5c8"
return size() > capacity;
```

---

17. Why `removeEldestEntry()` Is Useful

Without it, you'd need:

```text id="q8p8j2" insert ↓ check capacity ↓ find least-recently-used entry ↓ remove it

```
LinkedHashMap already maintains the ordering.

So:
```
```

```text id="k1z5lq"
access-order
+
removeEldestEntry()
```

gives you a simple LRU cache.

---

18. Is LinkedHashMap a Thread-Safe LRU Cache?

No.

This is an important trap.

This:

```java id="g6z3lq" LinkedHashMap
```

```
doesn't become thread-safe simply because it implements LRU behavior.

For concurrent applications, additional synchronization/concurrency design is
required.

---

# 19. LinkedHashMap vs HashMap

| Feature | HashMap | LinkedHashMap |
|---|---|---|
| Hash-based lookup | ✅ | ✅ |
| Average lookup | O(1) | O(1) |
| Ordering guarantee | ❌ | ✅ |
| Insertion order | ❌ | ✅ |
| Access order | ❌ | ✅ |
| Extra links | ❌ | ✅ |
| Memory overhead | Lower | Higher |
| Thread-safe | ❌ | ❌ |

The trade-off is:

```text id="q8j1t6"
Predictable ordering
```
```

```
        ↓
Additional memory / bookkeeping
```

---

20. LinkedHashMap vs TreeMap

Feature	LinkedHashMap	TreeMap
Structure	Hash table + links	Red-Black Tree
Ordering	Insertion/access	Sorted keys
Average lookup	O(1)	O(log n)
Key sorting	❌	✅
LRU support	✅	❌
Null key	One allowed	Generally not
Duplicate keys	❌	❌

Choose based on the required ordering semantics.

---

21. LinkedHashMap vs HashMap — When to Use Which?

Use:

```text id="8j7lqy"
HashMap
```

```
when:

> Ordering doesn't matter.

Use:

```text id="2f9eq8"
LinkedHashMap
```

when:

    You need predictable insertion order or access order.
```

Example:

```text id="u6d5nd" API response ↓ Preserve insertion order ↓ LinkedHashMap
```

```
---

# 22. LinkedHashMap and `entrySet()`

Iteration is predictable because the linked structure controls traversal.

```java id="7d2zj5"
for (Map.Entry<Integer, String> entry
        : map.entrySet()) {

    System.out.println(
        entry.getKey()
        + " = "
        + entry.getValue()
    );
}
```

The iteration follows the LinkedHashMap's ordering mode.

---

23. `keySet()` and `values()`

These views also reflect the map's ordering.

Example:

```java id="e6km5g" map.put(3, "C"); map.put(1, "A"); map.put(2, "B");
```

```
`keySet()` iterates predictably according to the LinkedHashMap order.

Likewise:

```java id="a2z2wl"
values()
```

follows the same entry ordering.

---

24. Does LinkedHashMap Sort Keys?

No.

This is an important distinction.

Suppose:

```java id="j3t9uq" map.put(30, "C"); map.put(10, "A"); map.put(20, "B");

```
LinkedHashMap gives:

```text id="3cvw9n"
30
10
20
```

not:

```text id="w2w1az" 10 20 30

```
If you need sorted keys:

```text id="r5v9mp"
TreeMap
```

---

25. Null Keys and Values

LinkedHashMap inherits HashMap's basic null behavior.

It permits:

```text id="p7d6m2" one null key multiple null values

```
Example:

```java id="9s4h11"
map.put(null, "A");
map.put(null, "B");
```

Final mapping:

```text id="42h7qs"
null → B
```

```
---

# 26. Capacity and Load Factor

LinkedHashMap supports the same basic capacity/load-factor configuration as
HashMap.

Example:

```java id="3t0g1n"
new LinkedHashMap<>(
    100,
    0.75f
);
```

The linked structure adds ordering information, but the underlying hash-table capacity/load-factor concepts remain relevant.

---

27. Access-Order Constructor

Memorize this constructor:

```java id="w7z8nq"
new LinkedHashMap<>( initialCapacity, loadFactor, accessOrder );
```

```
Example:

```java id="5a3h72"
new LinkedHashMap<>(
    16,
    0.75f,
    true
);
```

Meaning:

```text id="y13z0r"
16 → initial capacity
0.75 → load factor
true → access order
```

```
---

# 28. Default Access Order

If you use:

```java id="8xg4vy"
new LinkedHashMap<>();
```

the default is:

```text id="g72gsi" accessOrder = false
```

```
Therefore:

```text id="h4g0l1"
Insertion order
```

is the default.

---

29. Access Order Example

```java id="xxb6y1" LinkedHashMap<Integer, String> map = new LinkedHashMap<>( 16, 0.75f, true );

map.put(1, "A"); map.put(2, "B"); map.put(3, "C");
```

```
Current:

```text id="cqf4xe"
1 → 2 → 3
```

Access:

```java id="ypo5qm" map.get(2);
```

```
Now:
```
```

```text id="q0n8sv"
1 → 3 → 2
```

Access:

```java id="jv4c1a"
map.get(1);
```

```
Now:

```text id="t9c8th"
3 → 2 → 1
```

---

30. LRU Logic

With access order:

```text id="kwg9dt"
Head ↓ Least Recently Used ↓ ... ↓ Most Recently Used ↓ Tail
```

```
So:

```text id="4ng5h5"
eldest = least recently used
```

when the map is being used as an LRU cache.

This makes:

```java id="1h4w8j"
removeEldestEntry()
```

```
particularly useful.

---

# 31. LRU Example Step-by-Step

Capacity:

```text id="4r4yd0"
3
```
```

Insert:

```text id="b4m9xk"
A
B
C
```

````
Order:

```text id="j6f3sf"
A → B → C
```

Access A:

```text id="7p9nfc"
B → C → A
```

````
Access B:

```text id="0psnq8"
C → A → B
```

Insert D:

```text id="g0xj2z"
A → B → D
```

````
C is removed because it was least recently used.

---

# 32. `removeEldestEntry()` Subtlety

The method is called after a new mapping has been inserted.

Therefore the common condition:

```java id="3d6q0b"
return size() > capacity;
```

works naturally.

Example:

```text id="1a6a4v"
capacity = 3

insert 4th entry
↓
size = 4
↓
4 > 3
↓
remove eldest
```
```

```
---

# 33. Is LRU Cache O(1)?

The standard LinkedHashMap-based implementation provides approximately:

```text id="r7f1c5"
get() → O(1) average
put() → O(1) average
remove eldest → O(1)
```

This makes it a very attractive implementation for an in-memory LRU cache.

The actual performance still depends on hashing and implementation details.

---

# 34. Common Interview Question

### Why does LinkedHashMap maintain insertion order?

Answer:

> It maintains additional linked ordering information between entries, allowing iteration to follow insertion order independently of the hash-table bucket arrangement.

---

# 35. Common Interview Question

### How does LinkedHashMap achieve O(1) lookup while maintaining order?

Answer:

> It uses HashMap's hash-table mechanism for lookup and an additional doubly linked structure for ordering. Lookup and ordering are therefore handled by separate structures.

Excellent senior-level answer.

---
```

# 36. Common Interview Question

### What is the difference between insertion order and access order?

**Insertion order:**

```
Order entries were inserted.
```

**Access order:**

```
Order entries were most recently accessed.
```

Access order is especially useful for:

```
LRU cache
```

---

# 37. Common Interview Question

### Does `put()` move an existing key to the end?

Insertion-order mode:

```
No.
```

Access-order mode:

```
An operation that accesses an existing mapping can move it to the tail.
```

The exact behavior depends on the Map operation being used.

---

# 38. Common Interview Question

### Why use LinkedHashMap instead of TreeMap?

If you need:

```text id="c9e0v4" Insertion order
```

````
or:

```text id="d3k5m1"
Access order
````

use LinkedHashMap.

If you need:

```text id="7b7l0h" Sorted keys
```

````
use TreeMap.

---

# 39. Common Interview Question

### Is LinkedHashMap thread-safe?

No.

Like HashMap, it isn't inherently thread-safe.

---

# 40. Common Interview Question

### Can LinkedHashMap have duplicate keys?

No.

It follows Map semantics:

```text id="2u2b0q"
Key → unique
Value → duplicates allowed
````

---

41. Common Interview Question

Can LinkedHashMap contain null?

Yes.

It supports one null key and multiple null values.

---

42. ⭐ Coding Question

What is the output?

```java id="5o1h6e" Map<Integer, String> map = new LinkedHashMap<>();

map.put(3, "C"); map.put(1, "A"); map.put(2, "B");

for (Integer key : map.keySet()) { System.out.println(key); }
```

```
Output:

```text id="i2cb2f"
3
1
2
```

Because insertion order is maintained.

---

43. ⭐ Coding Question

What happens?

```java id="j7w1gr" LinkedHashMap<Integer, String> map = new LinkedHashMap<>();

map.put(1, "A"); map.put(2, "B"); map.put(3, "C");

map.put(2, "Updated");
```

```
Iteration order:
```
```

```text id="f6b3cv"
1
2
3
```

Updating an existing mapping does not move it in default insertion-order mode.

---

44. ⭐ Coding Question — Access Order

```java id="2j9h4k" LinkedHashMap<Integer, String> map = new LinkedHashMap<>( 16, 0.75f, true );

map.put(1, "A"); map.put(2, "B"); map.put(3, "C");

map.get(1);
```

Order:

```text id="y2h3cw"
2
3
1
```

Because `1` was accessed and moved to the end.

---

45. ⭐ Coding Question — LRU

```java id="1drj9y" LRUCache<Integer, String> cache = new LRUCache<>(2);

cache.put(1, "A"); cache.put(2, "B");

cache.get(1);

cache.put(3, "C");
```

Final entries:

```text id="5v5c7e"
```

```
1 → A
3 → C
```

Why?

Initial:

```text id="w3y7bm" 1 → 2

```
After `get(1)`:

```text id="r4j7t8"
2 → 1
```

Insert 3:

```text id="u5b1q6" 2 → 1 → 3

```
Capacity exceeded, so eldest `2` is removed:

```text id="2i6w6y"
1 → 3
```

---

46. Important Trap — Iteration Order

LinkedHashMap guarantees predictable iteration order.

But don't confuse:

```text id="y9f1km" Predictable

```
with:

```text id="n7k5b9"
Sorted
```

These are different.

Insertion order:

```text id="b7j4l8" 30 → 10 → 20

```
remains:

```text id="b7j4l8"
30 → 10 → 20
```

It doesn't become:

```text id="h0x2c7" 10 → 20 → 30

```
---

47. Important Trap — LRU and Thread Safety

This:

```java id="5d0nyq"
LinkedHashMap + accessOrder=true
```

does not automatically create a thread-safe LRU cache.

For multithreaded applications, you need an appropriate concurrency strategy.

---

48. Important Trap — `get()` and Modification

In access-order mode:

```java id="6qf2lz" map.get(key);

```
can reorder the linked list.

Therefore a `get()` can affect iteration behavior.

This is one of the most useful advanced LinkedHashMap interview details.

---

49. LinkedHashMap Memory Trade-Off

Compared with HashMap, each entry needs additional ordering references.
```
```

Conceptually:

```text id="4izbwm"
HashMap entry
→ key/value/hash/next

LinkedHashMap entry
→ key/value/hash/next
 + before/after
```

So:

```text id="e3yr6x" LinkedHashMap → more memory
in exchange for predictable ordering.

---

# 50. Decision Framework

```text id="n2k2k8"
Need Map
   ↓
Does ordering matter?
   ↓
 ┌─No──────────────┐
 ↓                 ↓
HashMap          Yes
                   ↓
             What ordering?
              ↙          ↘
       Insertion       Access
          ↓               ↓
   LinkedHashMap    LinkedHashMap
                       accessOrder=true
                           ↓
                       LRU cache
```

Sorted keys:

```text id="j4b7sp" Need sorted keys? ↓ TreeMap
```
```

```
Concurrent access:

```text id="n1k7b9"
Need concurrent Map?
      ↓
ConcurrentHashMap
```

---

51. HashMap vs LinkedHashMap vs TreeMap

This is worth memorizing for interviews:

```text id="q5u8v2" HashMap ↓ Fast lookup No ordering

LinkedHashMap ↓ Fast lookup Predictable insertion/access order

TreeMap ↓ Sorted keys O(log n)

```

🧠 Final Mental Model

```text id="9p6b3a"
                    LinkedHashMap
                          |
              ┌───────────┴────────────┐
              ↓                        ↓
        Hash Table              Linked List
              ↓                        ↓
       Fast lookup              Ordering
              |                        |
              └────────────┬───────────┘
                           ↓
                    Predictable
                      iteration
```

Default:

```text id="r5j7sx" LinkedHashMap ↓ Insertion order

```
```
Special mode:
```
```

```text id="x4r8p1"
accessOrder = true
      ↓
Access order
      ↓
LRU Cache
```

🔥 Chapter Summary

Concept	Key Point
LinkedHashMap	HashMap + ordering
Lookup	Hash-based
Ordering	Insertion or access
Default order	Insertion
Access order	Constructor flag `true`
Internal ordering	Doubly linked structure
Average `get()`	O(1)
Average `put()`	O(1)
Sorted keys	❌
Duplicates keys	❌
Duplicate values	✅
Null key	One allowed
Null values	Allowed
Thread-safe	❌
LRU cache	Excellent fit
`removeEldestEntry()`	Useful for bounded caches

🎯 Interview Readiness Checklist

You should be able to explain:

- [ ] How LinkedHashMap differs from HashMap

- [ ] How it maintains ordering
- [ ] Hash table + linked-list structure
- [ ] Insertion order
- [ ] Access order
- [ ] `accessOrder=true`
- [ ] Why `get()` can reorder entries
- [ ] `removeEldestEntry()`
- [ ] How to implement LRU cache
- [ ] LinkedHashMap vs HashMap
- [ ] LinkedHashMap vs TreeMap
- [ ] Why LinkedHashMap uses more memory
- [ ] Why it isn't thread-safe
- [ ] Why predictable order isn't the same as sorted order
- [ ] Complexity of common operations
- [ ] How existing-key updates behave in insertion-order mode

---

🚀 Next Chapter

Chapter 17 — TreeMap (Complete Deep Dive)

We'll focus on what's new:

- Red-Black Tree internals
- Sorted key ordering
- `Comparable` vs `Comparator`
- Natural ordering
- Custom ordering
- `put()` and tree insertion
- Rotations
- Rebalancing
- Why operations are O(log n)
- `firstKey()` / `lastKey()`
- `higherKey()` / `lowerKey()`
- `ceilingKey()` / `floorKey()`
- `subMap()` / `headMap()` / `tailMap()`
- NavigableMap
- Range queries
- TreeMap vs HashMap vs LinkedHashMap
- Null behavior
- Mutable-key/comparator pitfalls
- 20+ interview questions